Software construction and development Practical

Week 1

Object Oriented Software Development

Object-oriented software development is a programming methodology that focuses on creating software based on the concept of "objects." An object is an instance of a class, which is a blueprint that defines the data and behavior of the object.

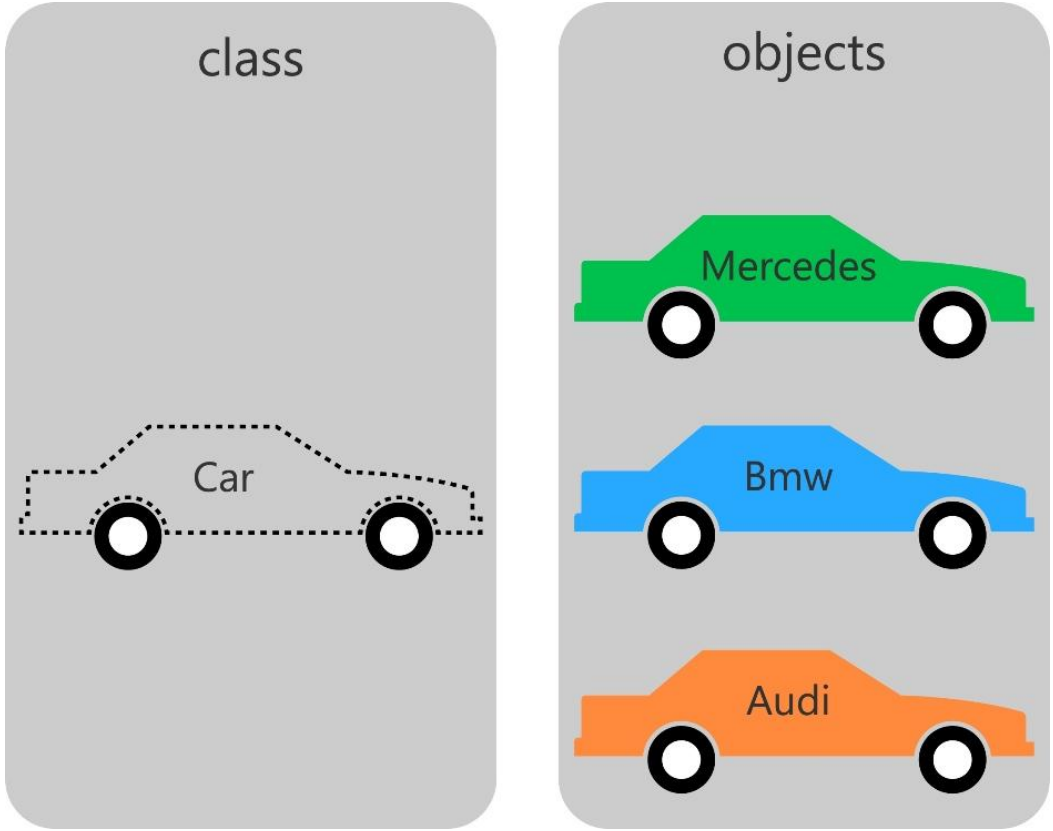
Principles of object-oriented software development

- **Encapsulation:** The concept of encapsulation involves bundling data and behavior together into a single unit, known as a class. The class hides the data and behavior from the outside world and provides an interface for interacting with the object.
- **Inheritance:** Inheritance is the process by which one class can inherit properties and behavior from another class. This enables developers to reuse code and avoid duplicating effort.
- **Polymorphism:** Polymorphism allows objects to take on multiple forms, meaning they can be used in different contexts with different behaviors.
- **Abstraction:** Abstraction is the process of simplifying complex systems by breaking them down into smaller, more manageable parts. In object-oriented software development, this is achieved through the use of abstract classes and interfaces.

Class and Object

- In object-oriented programming, a class is a blueprint for creating objects. A class defines a set of properties and behaviors that are common to all objects of that class. For example, a class might define the properties and behaviors of a car, such as its make, model, and color, as well as its ability to accelerate, brake, and turn.
- An object, on the other hand, is an instance of a class. When you create an object, you are creating a specific instance of a class, with its own unique set of properties and behaviors.

Class and Object



Object Properties

- Object properties are the characteristics or attributes of an object. These properties can be set when the object is created or modified later during the program's execution. For example, in the car class, the properties might include make, model, color, year, and mileage. These properties define the characteristics of the car object and can be used to determine how the car behaves in the program.
- In addition to their properties, objects also have behaviors or methods, which are actions that the object can perform. For example, the car object might have methods for accelerating, braking, turning, and changing gears. These methods define the actions that the object can perform and can be called by the program to control the behavior of the object.

Object Life Time

- In object-oriented programming, the term "**object lifetime**" refers to the period during which an object exists in memory. The lifetime of an object is determined by when it is created and when it is destroyed.
- The creation of an object occurs when the program allocates memory for the object and initializes its properties. The object remains in memory and can be accessed and modified by the program until it is explicitly destroyed or until the program terminates.
- The destruction of an object occurs when the program releases the memory allocated for the object. This typically happens when the object is no longer needed or when the program completes its execution.

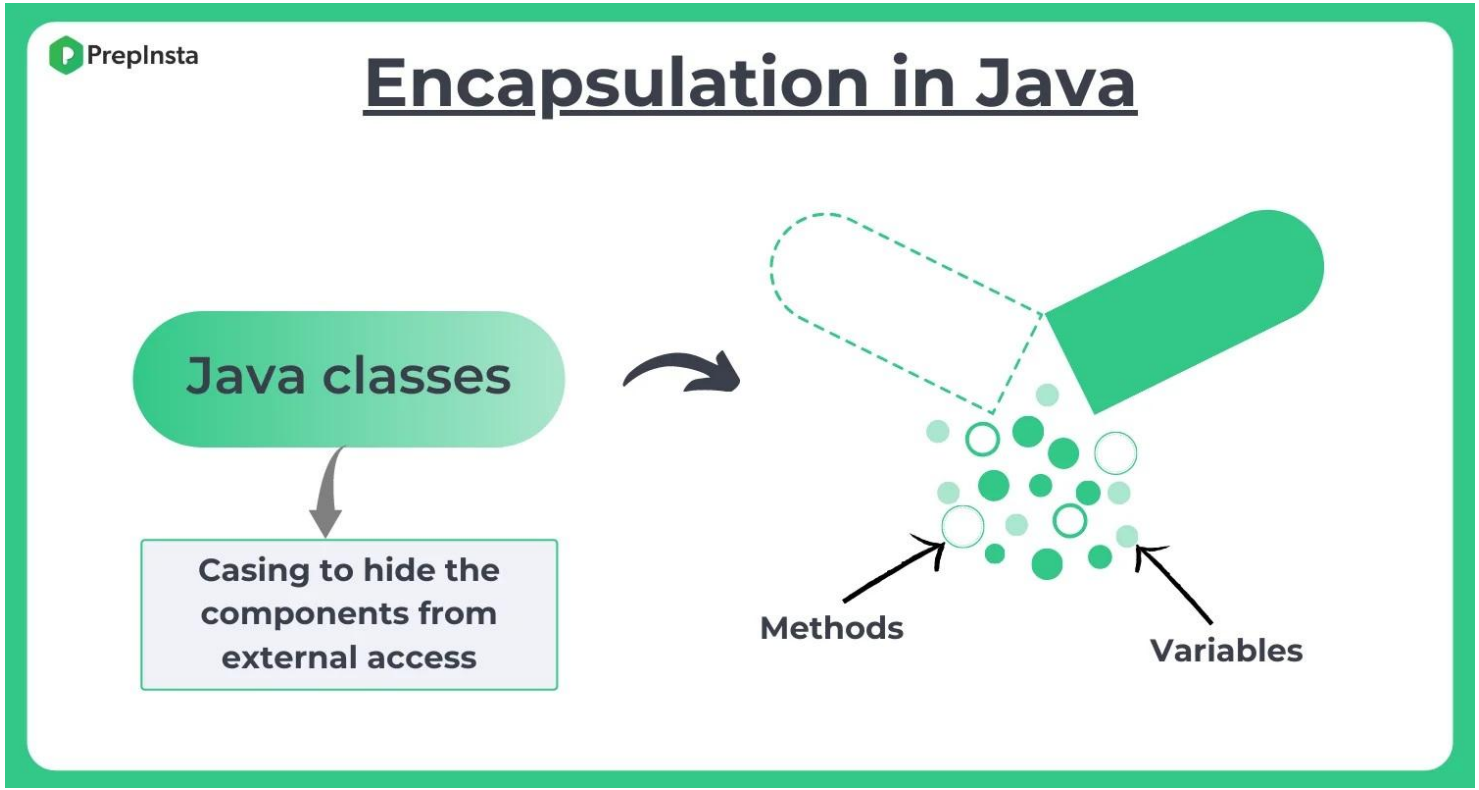
Information Hiding

Information hiding is a software design principle that aims to minimize the interdependence between different parts of a program by limiting the access to the internal details of an object or module. It involves encapsulating data and implementation details within a module or class, and exposing only the relevant interfaces or methods that can be used by other modules or classes. This helps to reduce complexity, increase security, and improve maintainability, as changes to the internal details of a module or class do not affect other parts of the program.

Encapsulation

Encapsulation is a software design principle that involves bundling data and behavior into a single unit, such as a class, and restricting access to the data and behavior from outside the unit. This helps to reduce the complexity of a program, increase security, and improve maintainability, as changes to the internal details of the unit do not affect other parts of the program. Encapsulation also enables the unit to provide a well-defined interface that other parts of the program can use to interact with the data and behavior it contains, while keeping the internal details hidden.

Encapsulation



Association

Association is a relationship between two or more objects in which they are connected through a shared reference to one another. It is one of the fundamental concepts in object-oriented programming and is often represented by a line connecting the related objects in a class diagram. Associations can be either one-to-one, one-to-many, or many-to-many, and can be either directional or bidirectional. The relationship between the objects in an association can be navigated in both directions, allowing one object to access the properties and methods of the other object.

Association UseCase

- One common use case of association in object-oriented programming is modeling the relationship between a customer and their orders in an e-commerce application.
- In this scenario, a customer object would have an association with one or more order objects. The association could be represented by a line connecting the two objects in a UML class diagram, and could be either directional (from the customer to the order) or bidirectional (allowing both the customer and the order to access each other's properties and methods).
- The association would enable the customer object to access the details of their orders (such as the order number, status, and items) and to perform actions on the orders (such as canceling or modifying them). Similarly, the order objects could access the details of the customer (such as their name and address) and use that information to process the order.

Composition

Composition is a design pattern in object-oriented programming where an object is composed of one or more other objects, and the composed objects cannot exist independently without the main object. The composed objects are typically created and destroyed along with the main object.

Composition use case

- A common use case for composition is modeling a car, where a car is composed of various parts such as an engine, wheels, doors, and seats. The car object is the main object, and the parts are the composed objects. The parts cannot exist independently without the car.
- In this scenario, the car object would have a composition relationship with the various parts objects. The parts would be created and destroyed along with the car object, and the car object would provide an interface for accessing and controlling the parts.
- For example, the car object could have a method for starting the engine, which in turn would start the composed engine object. The car object could also have methods for opening and closing the doors, which would manipulate the composed door objects. The parts objects would be encapsulated within the car object, and could not be accessed or manipulated directly by the rest of the program. This would help to simplify the code, improve maintainability, and reduce the risk of errors.

Aggregation

Aggregation is a design pattern in object-oriented programming where an object is composed of one or more other objects, but the composed objects can exist independently without the main object. The composed objects are typically created and destroyed independently of the main object.

Aggregation use case

- A common use case for aggregation is modeling a school, where a school is composed of various departments such as the math department, science department, and history department. The department objects can exist independently without the school object.
- In this scenario, the school object would have an aggregation relationship with the department objects. The department objects would be created and destroyed independently of the school object, and could be shared by multiple school objects.
- For example, a teacher object could have an aggregation relationship with a department object, indicating that the teacher belongs to a specific department. The department object could have a list of teacher objects that belong to that department. The teacher objects could be shared by multiple department objects if the teachers teach multiple subjects.
- The aggregation relationship would allow the school object to access and manipulate the department objects, but the department objects could also be accessed and manipulated independently by other parts of the program. This would provide flexibility and allow for greater reusability of the department objects.

Software construction and development Practical

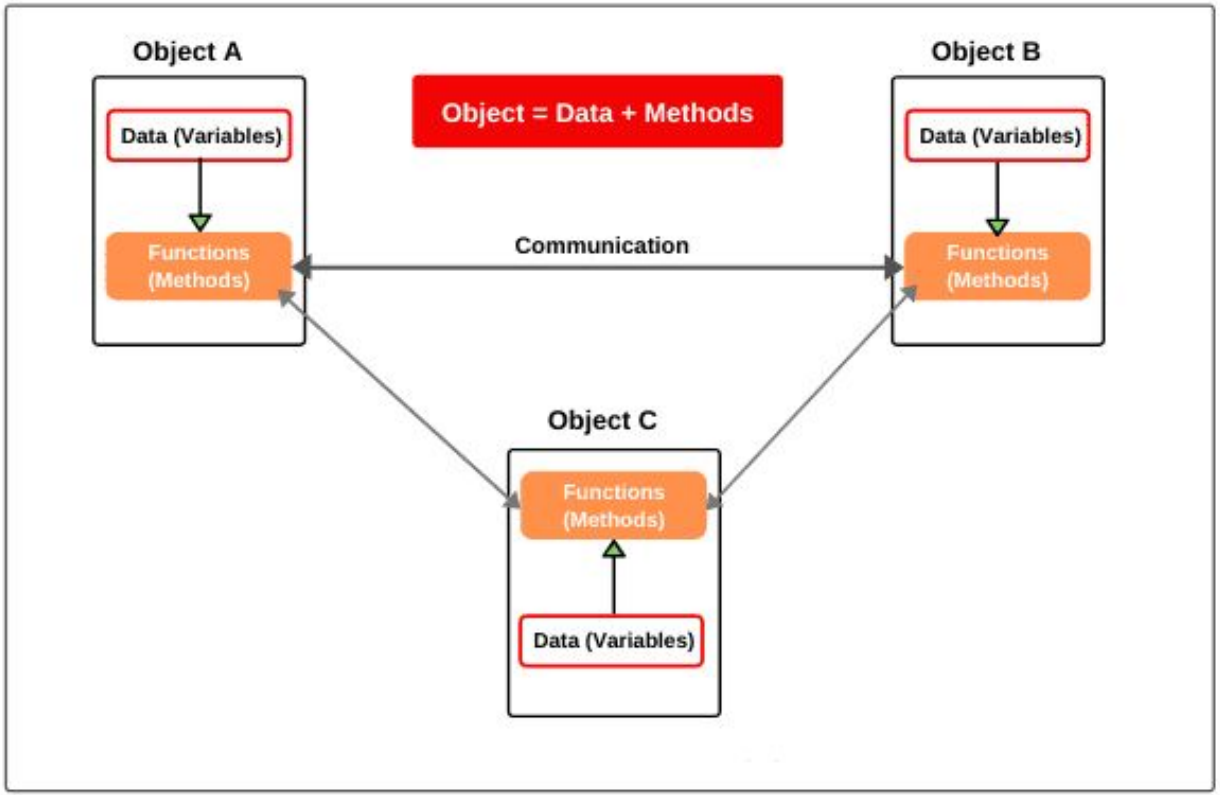
Week 2

Information sharing between objects

In object-oriented programming, objects can share information with each other through various means, such as method calls, parameter passing, and message passing.

- **Method calls** allow one object to call a method of another object, passing any necessary parameters as arguments. This allows the receiving object to use the information provided by the calling object to perform some action.
- **Parameter passing** allows one object to pass data directly to another object, such as passing an object reference to a method. This allows the receiving object to use the data provided by the calling object to perform some action.
- **Message passing** is another way for objects to communicate with each other. In this case, one object sends a message to another object, which can then process the message and take some action based on its contents.

Data Sharing- UseCase



Generalization

Generalization in object-oriented programming is a mechanism that allows classes to inherit properties and behavior from other classes. Here are some key points to understand about generalization:

- Generalization allows for the creation of new classes that are based on existing classes, but with additional features or behavior.
- The existing class is called the superclass, while the new class is called the subclass.
- Subclasses inherit properties and methods from their superclass, but they can also add new properties and methods or override existing ones.
- Generalization allows for the creation of hierarchies of classes, with increasingly specific subclasses inheriting from more general superclasses.
- Generalization can improve the efficiency of code development and maintenance, as it reduces the need to duplicate code across multiple classes.

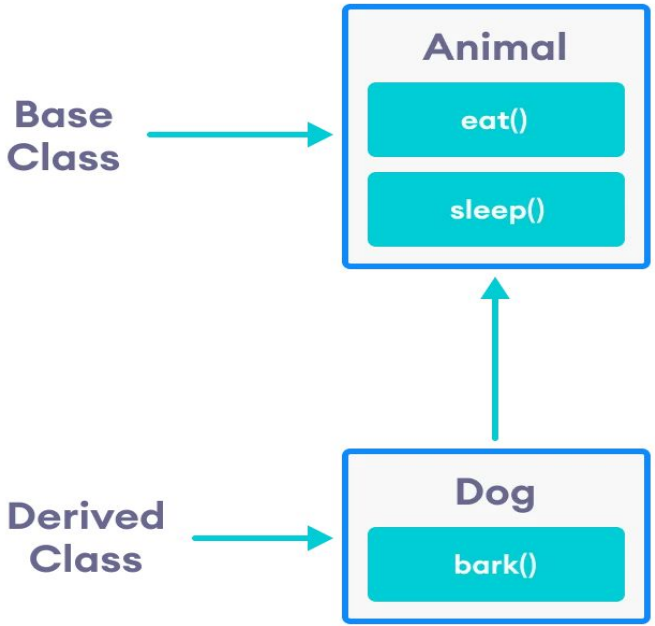
Inheritance, Extension, Overriding

- **Inheritance:** Inheritance is a mechanism that allows a new class to be based on an existing class. The new class, called the subclass, inherits all the properties and methods of the existing class, called the superclass. Inheritance allows for the creation of hierarchical structures of classes, where each subclass is more specific than its superclass.
- **Extension:** Extension is the process of adding new properties and methods to a subclass that are not present in its superclass. This allows the subclass to have additional features or behavior that are not shared by its superclass.
- **Overriding:** Overriding is the process of redefining a property or method in a subclass that is already present in its superclass. This allows the subclass to modify the behavior of the inherited property or method to suit its own specific needs.

Inheritance usecase

```
class Animal {
    // eat() function
    // sleep() function
};

class Dog : public Animal {
    // bark() function
  };
```



Realization

Realization is a concept in object-oriented programming that allows a class to implement the functionality defined by an interface. Here are some key points to understand about realization:

- An interface is a collection of abstract methods that define the behavior of a class.
- A class can realize an interface by implementing all of the methods defined in the interface.
- Realization allows for the creation of modular and flexible code, as it allows classes to share behavior without having to share implementation details.
- Realization is an important part of the design of large, complex systems, as it allows for easy integration and extension of existing code.
- Realization is often used in combination with other object-oriented programming concepts, such as inheritance and composition, to create complex, flexible systems that can be easily modified and extended.

Realization usecase

```
// Drivable.java
public interface Drivable {
   public void drive();

public class Car implements Drivable {
    // Must implements all functions of Drivable
   public void drive() {
        // Do the driving here
    } }
```

Polymorphism

Polymorphism in object-oriented programming allows objects to take on **multiple forms** and be treated as different types depending on the context.

- This can take many forms, including method overloading, method overriding, and interfaces.
- Method overloading allows a class to define multiple methods with the same name but different parameters, while method overriding allows a subclass to redefine the implementation of an inherited method.
- Interfaces allow for the creation of multiple classes that share a common set of methods but with different implementations. These concepts allow for the creation of flexible and modular code that can be easily modified and extended.

Polymorphism-UseCase

```
class Vehicle {
  public void move() {
    System.out.println("The vehicle moves");    }}

class Car extends Vehicle {
  public void move() {
    System.out.println("The car drives on the road");  }}
class Plane extends Vehicle {
  public void move() {
    System.out.println("The plane flies through the air");  }}
```

```
class Main {
  public static void main(String[] args) {
    Vehicle vehicle1 = new Vehicle();
    Vehicle vehicle2 = new Car();
    Vehicle vehicle3 = new Plane();
    vehicle1.move();
    vehicle2.move();
    vehicle3.move();
  }
}
```

Software construction and development Practical

Week 3

Generic Programming

- Generic programming is a way to write code that is **reusable** across different data types.
- It allows you to write functions and classes that can work with a variety of **different data types**, without needing to rewrite the code for each type.
- Generic programming is often used in languages like Java and C++ to create container classes like **lists and maps** that can hold objects of any type.
- Generics are often implemented using **type** parameters.
- By writing generic code, you can make your programs more flexible and easier to maintain over time.

```
class Test<T> {
    // An object of type T is declared
    T obj;
    Test(T obj) {
     this.obj = obj;
                 } // constructor
    public T getObject() {  return this.obj; }
}
```

```
 Test<Integer> iObj =  new Test<Integer>(15);
```

Constants and good programming practices

- **Constants** are values in a program that do not change and are defined as final or const variables.
- Constants should be used for values that will not change during runtime, such as **mathematical constants** or program configurations.
- Using constants makes code easier to read and understand, as it is clear which values are intended to be static.
- Good programming practices suggest that constants should be declared in all **uppercase letters** with underscores separating words, to clearly differentiate them from regular variables.
- Constants should be placed in a central location within a program so that they are easily accessible and can be modified if necessary.
- Constants should be used sparingly and not to represent every numerical or string value, as **overuse** can make code more difficult to understand.

Object Model Case Study



A university has a student management system that tracks student enrollment, grades, and other information. The system is built using object-oriented programming principles. The key objects in the system are **Student**, **Course**, and **Enrollment**. Each Student object has a **name**, **student ID**, and list of enrolled Course objects. Each Course object has a course ID, name, and list of enrolled Student objects. Each Enrollment object represents a student's enrollment in a course and contains a reference to the Student and Course objects involved.

Good Programming Practices

- Write code that is **easy to read** and understand, even by someone who didn't write it.
- Use descriptive and **meaningful variable names** that accurately reflect the data they represent.
- Break code into logical **functions and classes** that do one thing well.
- Keep functions and **classes small** and focused on a single task.
- Use **comments** to explain complex logic and document code in a way that is easy to understand.
- Test code thoroughly to identify and **fix bugs early** in the development process.
- Regularly **refactor code** to remove duplication and improve its maintainability and readability.
- Follow **industry-standard** coding conventions and style guidelines to ensure consistency and readability across codebases.

Distributed Objects

- Distributed Objects are objects that are designed to run on different systems in a distributed computing environment.
- They communicate with each other using remote procedure calls (RPCs) or messaging protocols such as **HTTP, SOAP,** or **REST.**
- Distributed Objects allow systems to be built using a modular and scalable architecture, where components can be distributed across multiple machines.
- They provide a way to share resources and functionality across a distributed network, allowing applications to be built that are more fault-tolerant and can scale horizontally as needed.
- Distributed Objects can be implemented using a variety of technologies, such as **CORBA**, **Java RMI, .NET Remoting**, and **Web Services.**

Object Serialization

- Object Serialization is the process of converting an object's state to a **byte stream** so that it can be stored, **transmitted**, or reconstructed at a later time.
- It is used in distributed computing, persistence, caching, and messaging systems to enable the transfer of objects between different systems or processes.
- Serialization involves converting an object into a stream of bytes, which can then be transmitted over a network or stored in a file.
- Deserialization is the process of converting a byte stream back into an object in memory.
- Serialization can be done in different formats, such as **Java Serialization**, **JSON**, **XML**, and Protocol Buffers.
- Object Serialization is important for maintaining the state of an object and preserving its identity, even when it is transferred or stored in a different location or system.

Peer to Peer Network-UseCase

A use case for peer-to-peer (P2P) networks that involves object serialization might be file sharing between two peers on the network. Here's how the process might work:

- Peer A has a file that they want to share with Peer B.
- Peer A serializes the file object into a byte stream and sends it over the P2P network to Peer B.
- Peer B receives the byte stream and deserializes it into a file object in memory.
- Peer B can then save the file to their local disk or perform any other operations on it.

By using object serialization, the file can be easily transmitted over the P2P network, and the receiver can reconstruct the file object in memory without losing any of its state or metadata. This allows for efficient and secure sharing of files between peers on a distributed network.

Software construction and development Practical

Week 4

Exception handling

Exception handling is a programming concept that deals with the occurrence of unexpected errors during program execution. It involves detecting, analyzing, and responding to these errors in a way that prevents the program from crashing and provides useful information to the user or developer.

Exception handling Code

```
try {

   // Code that may throw an exception

   int result = 10 / 0; // This will throw an
ArithmeticException

} catch (ArithmeticException e) {

   // Code to handle the exception

   System.out.println("An error occurred: " +
e.getMessage());

} finally {

   // Code that will always run, regardless of
whether an exception was thrown or not

   System.out.println("This code will always run.");

}
```

In this example, we have a try block that contains some code that may throw an exception (in this case, we are dividing by zero, which will throw an ArithmeticException). We then have a catch block that specifies the type of exception we want to catch (in this case, ArithmeticException) and some code to handle the exception (we are just printing an error message here). Finally, we have a finally block that contains code that will always run, regardless of whether an exception was thrown or not.

Exception handling VS Error handling

Exception Handling

```
try {

    int result = 10 / 0; // This will throw an ArithmeticException

} catch (ArithmeticException e) {

    System.out.println("An error occurred: " + e.getMessage());

}
```

Error Handling

```
try {

    int[] arr = new int[1000000000]; // This will throw an OutOfMemoryError

} catch (OutOfMemoryError e) {

    System.out.println("An error occurred: " + e.getMessage());

}
```

System vs Business Exception

System Exception

```
try {

    FileReader file = new FileReader("file.txt"); // This will throw a
FileNotFoundException, which is a system exception

    BufferedReader reader = new BufferedReader(file);

    String line = reader.readLine();

    System.out.println(line);

    reader.close();

} catch (FileNotFoundException e) {

    System.out.println("The file could not be found: " + e.getMessage());

} catch (IOException e) {

    System.out.println("An I/O error occurred: " + e.getMessage());

}
```

Business Exception

```
public class WithdrawalException extends Exception {

    public WithdrawalException(String message) {

        super(message);   }   }

public class BankAccount {

    private double balance;

    public void withdraw(double amount) throws WithdrawalException {

        if (amount > balance) {

            throw new WithdrawalException("Insufficient funds.");

        }        balance -= amount;    } }
```

Handling System Exceptions

- System exceptions in Java are **predefined exceptions** that occur due to problems in the system or the environment in which the program is running.
- Examples of system exceptions include FileNotFoundException, IOException, ClassNotFoundException, OutOfMemoryError, and NullPointerException.
- To handle system exceptions, you can use the try-catch block.
- When an exception occurs, the runtime system creates an object of the exception class and throws it.
- You can use multiple catch blocks to handle different types of exceptions that may be thrown in the try block.
- You can also use the finally block to specify code that should be executed regardless of whether an exception is thrown or not.

User Defined Exceptions

```
1.  public class NegativeNumberException
    extends Exception {
2.      public NegativeNumberException(String
    message) {
3.          super(message);
4.      }   }
5.  public class Calculator {
6.      public static int divide(int a, int b)
    throws NegativeNumberException {
7.          if (b < 0) {
8.              throw new
    NegativeNumberException("Cannot divide by
    a negative number.");
9.          }
10.         return a / b;
11.     }   }
```

```
12. public class Main {
13.     public static void main(String[]
    args) {
14.         try {
15.             int result =
    Calculator.divide(10, -2);
16.
    System.out.println("Result: " +
    result);
17.         } catch
    (NegativeNumberException e) {
18.             System.out.println("An
    error occurred: " + e.getMessage());
19.         }
20.     }
21. }
```

Software construction and development Practical - I

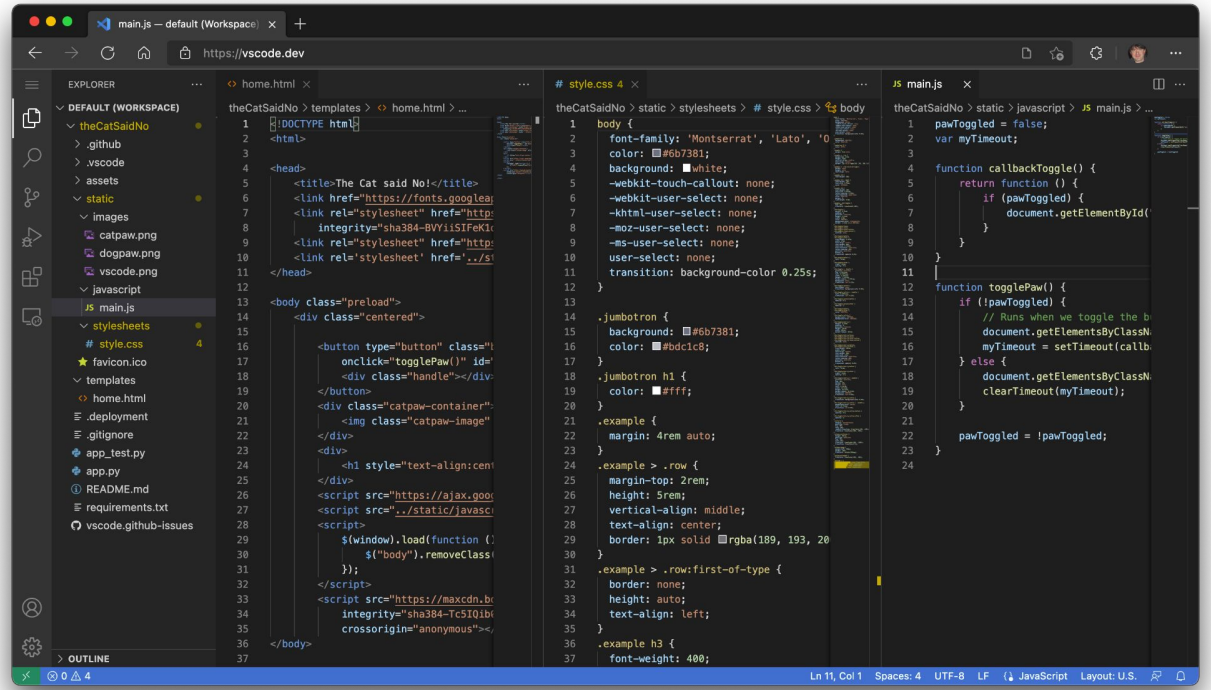
Week 5

Integrated Development Environments (IDEs)

These are software applications that provide a comprehensive environment for software development.

- Eclipse
- IntelliJ
- Visual Studio

Visual Studio Code

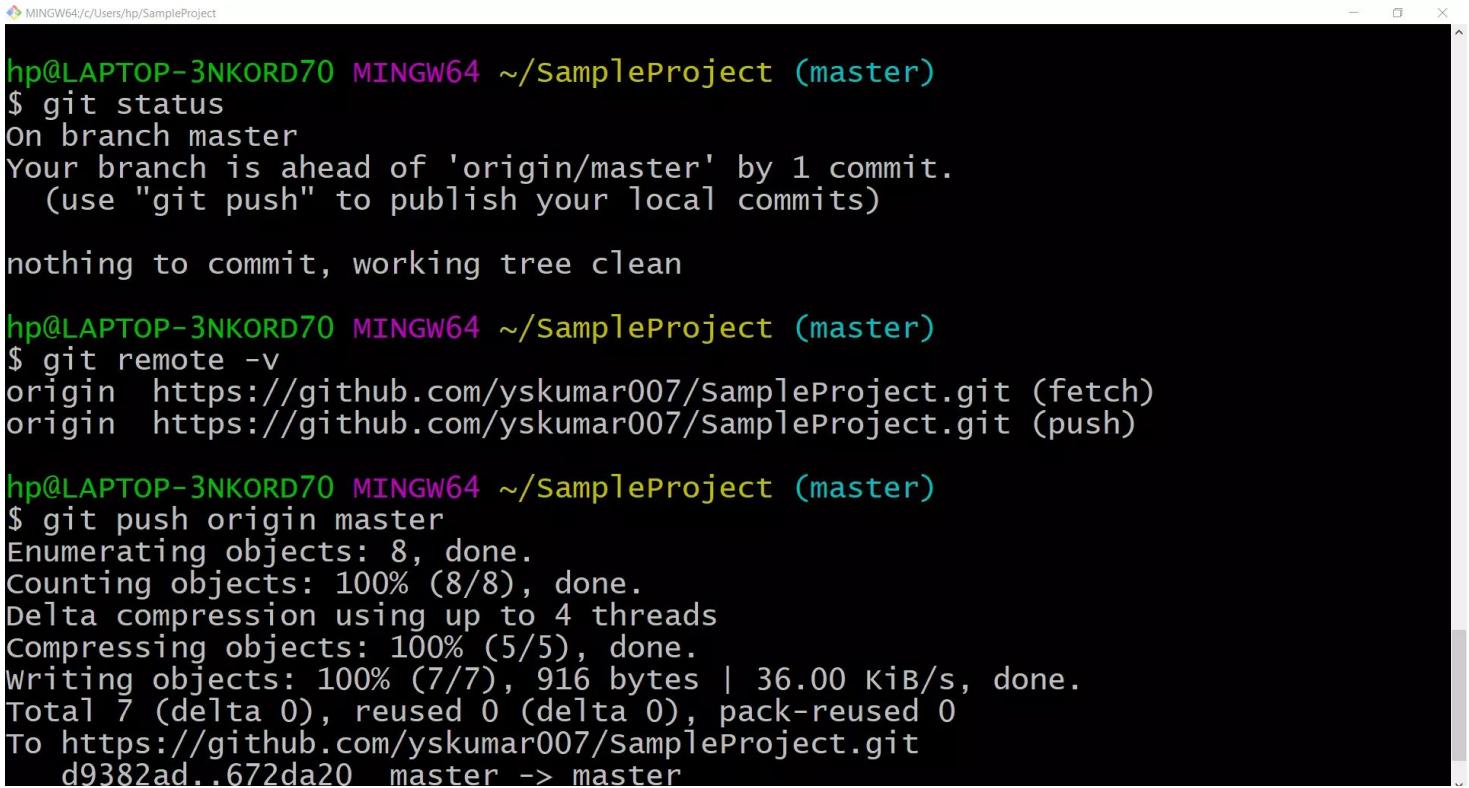


Version Control Systems (VCS)

These are tools used to manage changes to source code over time.

- Git
- SVN
- Mercurial

Git Bash


MINGW64:/c/Users/hp/SampleProject
hp@LAPTOP-3NKORD70 MINGW64 ~/SampleProject (master)
$ git status
On branch master
Your branch is ahead of 'origin/master' by 1 commit.
(use "git push" to publish your local commits)
nothing to commit, working tree clean
hp@LAPTOP-3NKORD70 MINGW64 ~/SampleProject (master)
$ git remote -v
origin https://github.com/yskumar007/SampleProject.git (fetch)
origin https://github.com/yskumar007/SampleProject.git (push)
hp@LAPTOP-3NKORD70 MINGW64 ~/SampleProject (master)
$ git push origin master
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 4 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 916 bytes | 36.00 KiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/yskumar007/SampleProject.git
d9382ad..672da20 master -> master

Build Automation Tools

These tools automate the process of building and packaging software applications.

- Maven
- Gradle
- Ant

Android Studio Gradle

```
implementation 'com.squareup.retrofit2:retrofit:2.9.0'
implementation 'com.google.code.gson:gson:2.8.0'
implementation 'com.squareup.retrofit2:converter-gson:2.3.0'

implementation 'com.github.ibrahimsn98:CirclesLoadingView:1.0'

implementation "com.google.android.gms:play-services-location:19.0.1"

compile 'org.osmdroid:osmdroid-android:6.1.11'


implementation 'com.github.alexfu:CountDownView:0.1.0' //Countdown Timer

def lottieVersion : String = "5.2.0"
implementation "com.airbnb.android:lottie:$lottieVersion"

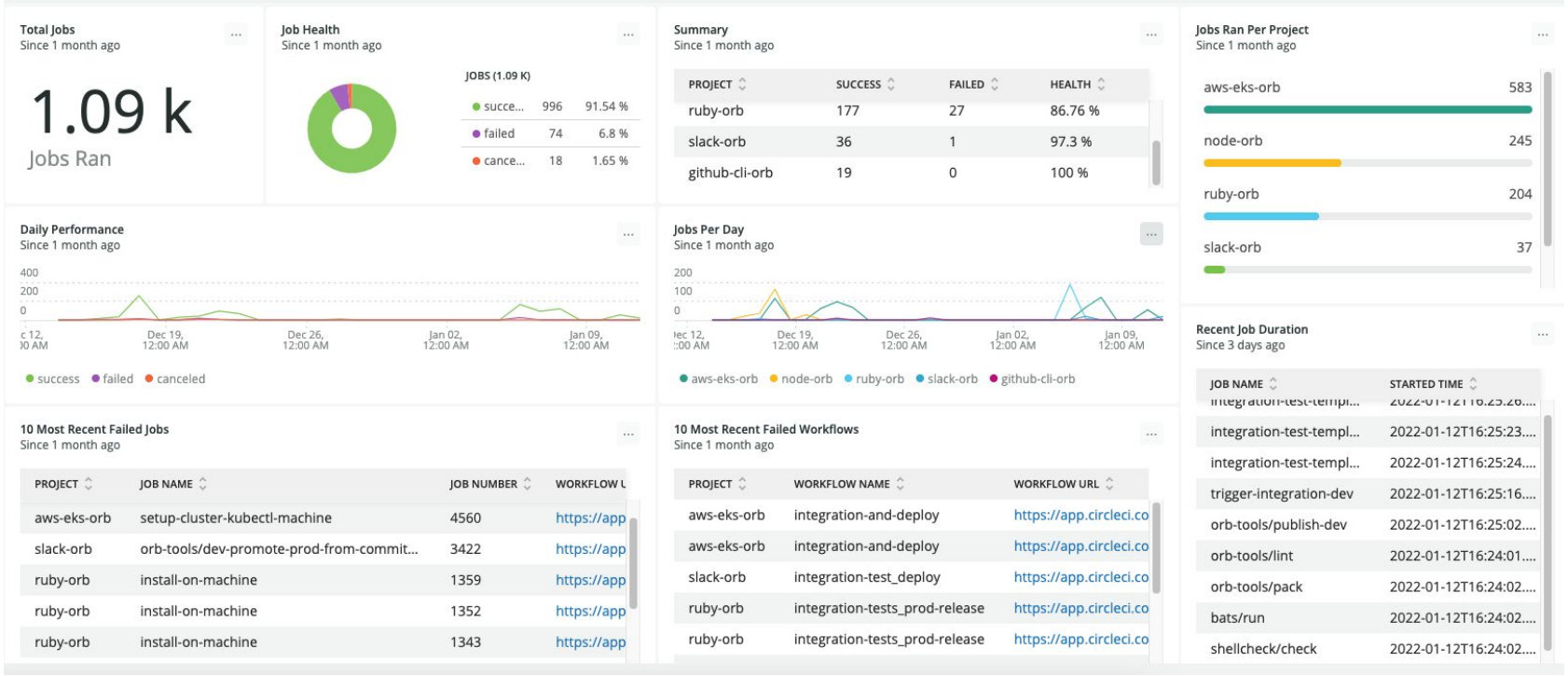
implementation 'commons-io:commons-io:2.4'
```

Continuous Integration and Continuous Delivery (CI/CD) Tools

These are tools used to automate the process of building, testing, and deploying software applications.

- Jenkins
- Travis CI
- CircleCI

CircleCI


Total Jobs
Since 1 month ago
1.09 k
Jobs Ran
Job Health
Since 1 month ago
JOBS (1.09 K)
succe... 996 91.54 %
failed 74 6.8 %
cance... 18 1.65 %
Summary
Since 1 month ago
PROJECT SUCCESS FAILED HEALTH
ruby-orb 177 27 86.76 %
slack-orb 36 1 97.3 %
github-cli-orb 19 0 100 %
Jobs Ran Per Project
Since 1 month ago
aws-eks-orb 583
node-orb 245
ruby-orb 204
slack-orb 37
Daily Performance
Since 1 month ago
400
200
0
Dec 19, 12:00 AM
Dec 26, 12:00 AM
Jan 02, 12:00 AM
Jan 09, 12:00 AM
success failed canceled
Jobs Per Day
Since 1 month ago
200
100
0
Dec 19, 12:00 AM
Dec 26, 12:00 AM
Jan 02, 12:00 AM
Jan 09, 12:00 AM
aws-eks-orb node-orb ruby-orb slack-orb github-cli-orb
Recent Job Duration
Since 3 days ago
JOB NAME STARTED TIME
integration-test-templ... 2022-01-12T16:25:23....
integration-test-templ... 2022-01-12T16:25:24....
trigger-integration-dev 2022-01-12T16:25:16....
orb-tools/publish-dev 2022-01-12T16:25:02....
orb-tools/lint 2022-01-12T16:24:01....
orb-tools/pack 2022-01-12T16:24:02....
bats/run 2022-01-12T16:24:02....
shellcheck/check 2022-01-12T16:24:02....
10 Most Recent Failed Jobs
Since 1 month ago
PROJECT JOB NAME JOB NUMBER WORKFLOW U
aws-eks-orb setup-cluster-kubectl-machine 4560 https://app
slack-orb orb-tools/dev-promote-prod-from-commit... 3422 https://app
ruby-orb install-on-machine 1359 https://app
ruby-orb install-on-machine 1352 https://app
ruby-orb install-on-machine 1343 https://app
10 Most Recent Failed Workflows
Since 1 month ago
PROJECT WORKFLOW NAME WORKFLOW URL
aws-eks-orb integration-and-deploy https://app.circleci.co
aws-eks-orb integration-and-deploy https://app.circleci.co
slack-orb integration-test_deploy https://app.circleci.co
ruby-orb integration-tests_prod-release https://app.circleci.co
ruby-orb integration-tests_prod-release https://app.circleci.co

Software construction and development Practical - I

Week 6

Code Review

- Code review is a critical part of the software development process, where developers evaluate and inspect each other's code to identify issues, suggest improvements, and ensure that the code is maintainable, scalable, and adheres to coding standards.
- Code review tools are software tools that automate and streamline the code review process, making it easier for developers to identify and fix issues, collaborate, and share feedback. These tools can help teams catch bugs and errors early in the development process, improve code quality, reduce technical debt, and ensure that the codebase is consistent and maintainable over time.

Code Review Tools

1. GitHub: A web-based code hosting platform that includes a built-in pull request review system for code review and collaboration.
2. Bitbucket: A web-based code hosting platform that provides a built-in code review system and integrates with Jira for issue tracking.
3. GitLab: A web-based Git repository management tool that provides a built-in code review system, continuous integration, and deployment features.
4. Crucible: A code review tool from Atlassian that provides a web-based interface for reviewing code changes and tracking issues.
5. CodeClimate: A code analysis tool that automatically identifies issues, technical debt, and code smells in the codebase and provides insights into code quality.
6. Review Board: A web-based code review tool that integrates with Git, Mercurial, and Subversion repositories and provides features for commenting, tracking issues, and automating the review process.

Bug Tracking Tools

- Bug Tracking Tools help software development teams track and manage issues, bugs, and defects in software applications through a central interface.
- They integrate with other software development tools to provide a streamlined workflow.
- These tools allow teams to identify and fix issues early in the development process, reducing the likelihood of bugs making it to production.
- Features of Bug Tracking Tools often include issue tracking, bug reporting, issue prioritization, status tracking, and issue assignment.
- Popular Bug Tracking Tools include Jira, Bugzilla, Mantis, and Redmine, among others.
- These tools are essential for large development teams working on complex projects to improve the quality and stability of their software applications.
- Many Bug Tracking Tools also include reporting and analytics features, providing teams with valuable insights into issue trends and bug severity.

Examples

- Popular Bug Tracking Tools include:
- Jira
- Bugzilla
- Mantis
- Redmine

Software Documentation Tools

- Software Documentation Tools help software development teams create, manage, and publish software documentation through a centralized platform.
- These tools make it easy for teams to create and update technical documents, user manuals, API documentation, and other types of software documentation.
- Features of Software Documentation Tools often include templates, version control, collaboration features, and the ability to embed images and multimedia content.
- These tools help teams keep their software documentation up-to-date and accessible to all stakeholders, including developers, project managers, and end-users.
- They provide a central platform for teams to collaborate on documentation, reducing the likelihood of siloed knowledge.Software Documentation Tools can also improve communication within the development team and with external stakeholders by providing a clear and consistent source of information about the software application.

Examples

Popular Software Documentation Tools include:

- Confluence
- Read the Docs
- Sphinx
- Jekyll

Testing Tools

- Testing Tools help software development teams automate and manage the testing process for software applications.
- These tools can be used to test functionality, performance, security, and other aspects of the software application.
- Features of Testing Tools often include test case management, test automation, and reporting capabilities.
- These tools can save time and improve the accuracy of testing, as well as reduce the need for manual testing.
- They can also improve collaboration between developers and testers by providing a common platform for test planning, execution, and reporting.
- Testing Tools can help teams identify and fix issues early in the development process, reducing the likelihood of bugs making it to production.
- They can also help teams ensure compliance with industry standards and regulations, such as ISO and GDPR.
- Overall, Testing Tools are essential for ensuring the quality and reliability of software applications, improving user satisfaction, and reducing the risk of negative business impact.

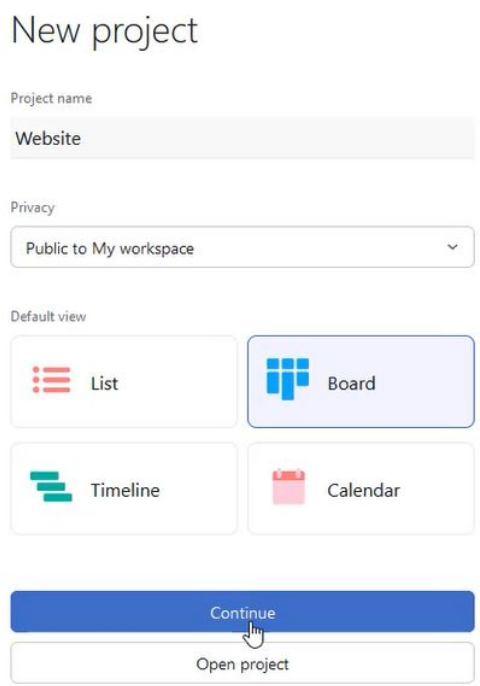
Examples

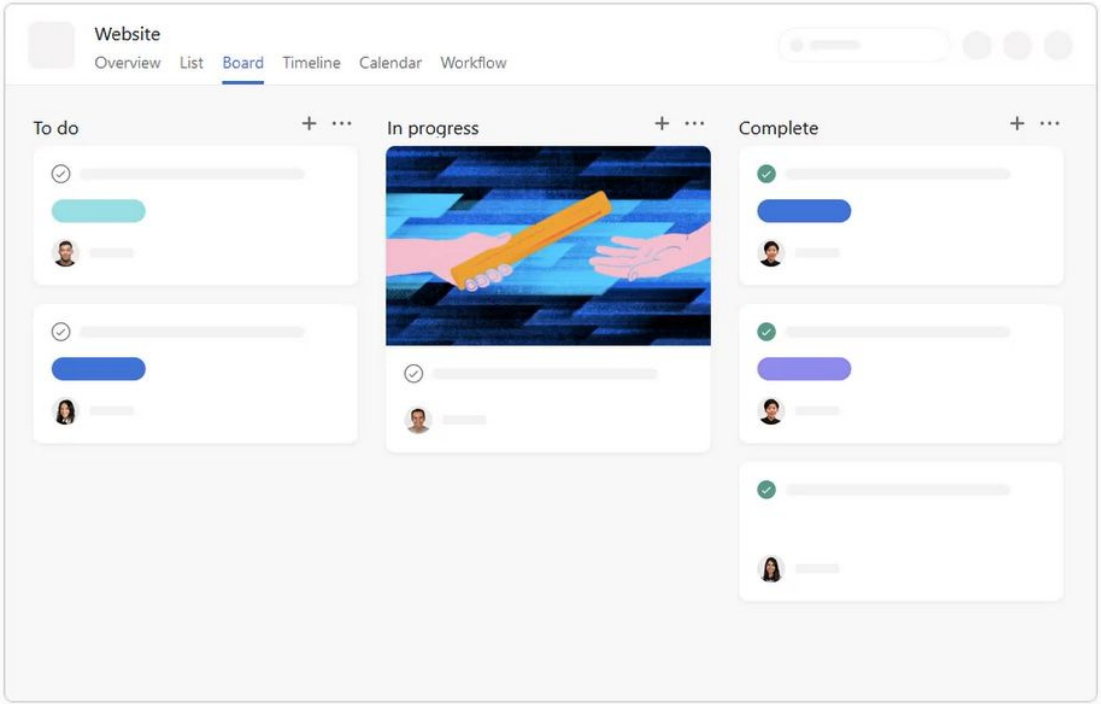
- Selenium
- JMeter
- Postman
- SoapUI
- TestRail

Software construction and development Practical - I

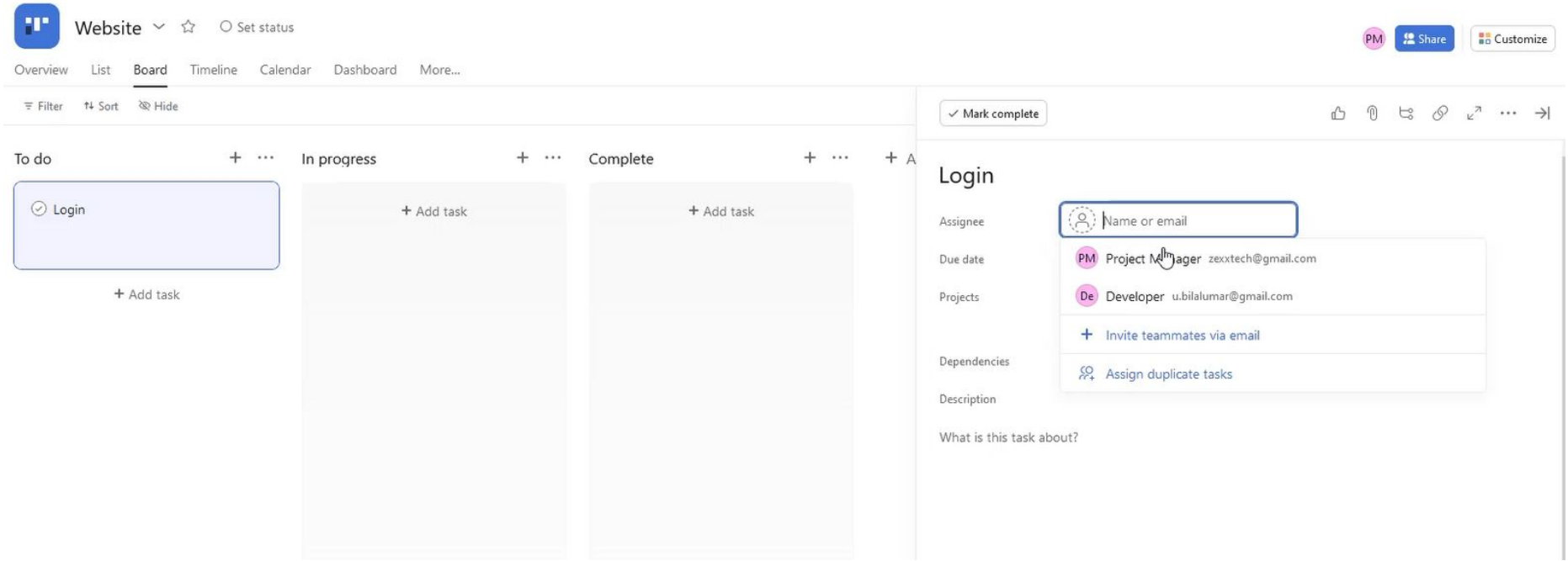
Week 8

Create a new project in Asana

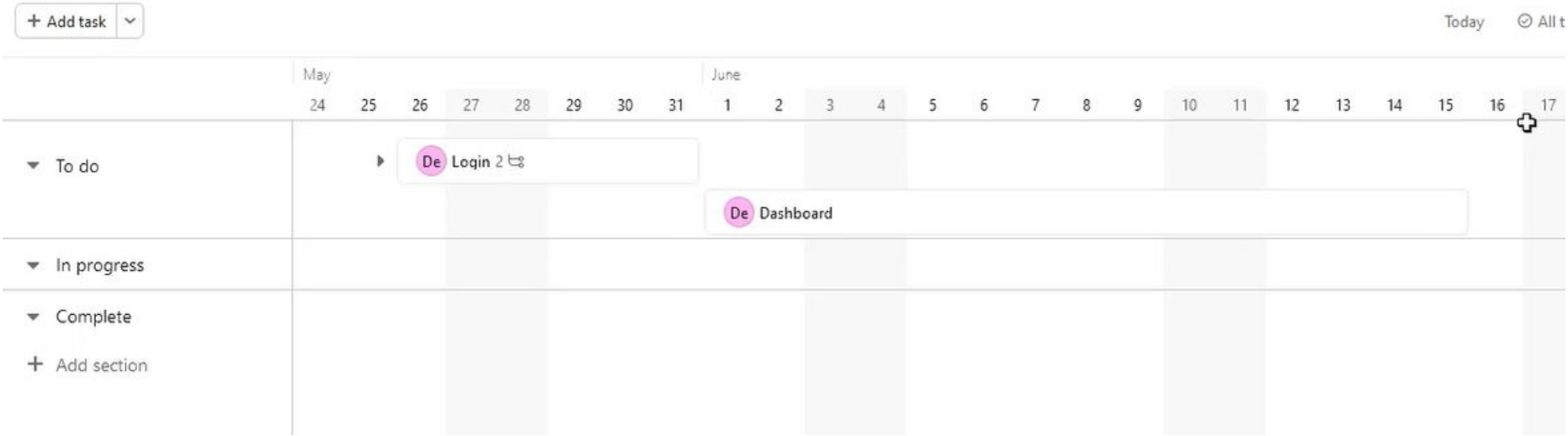




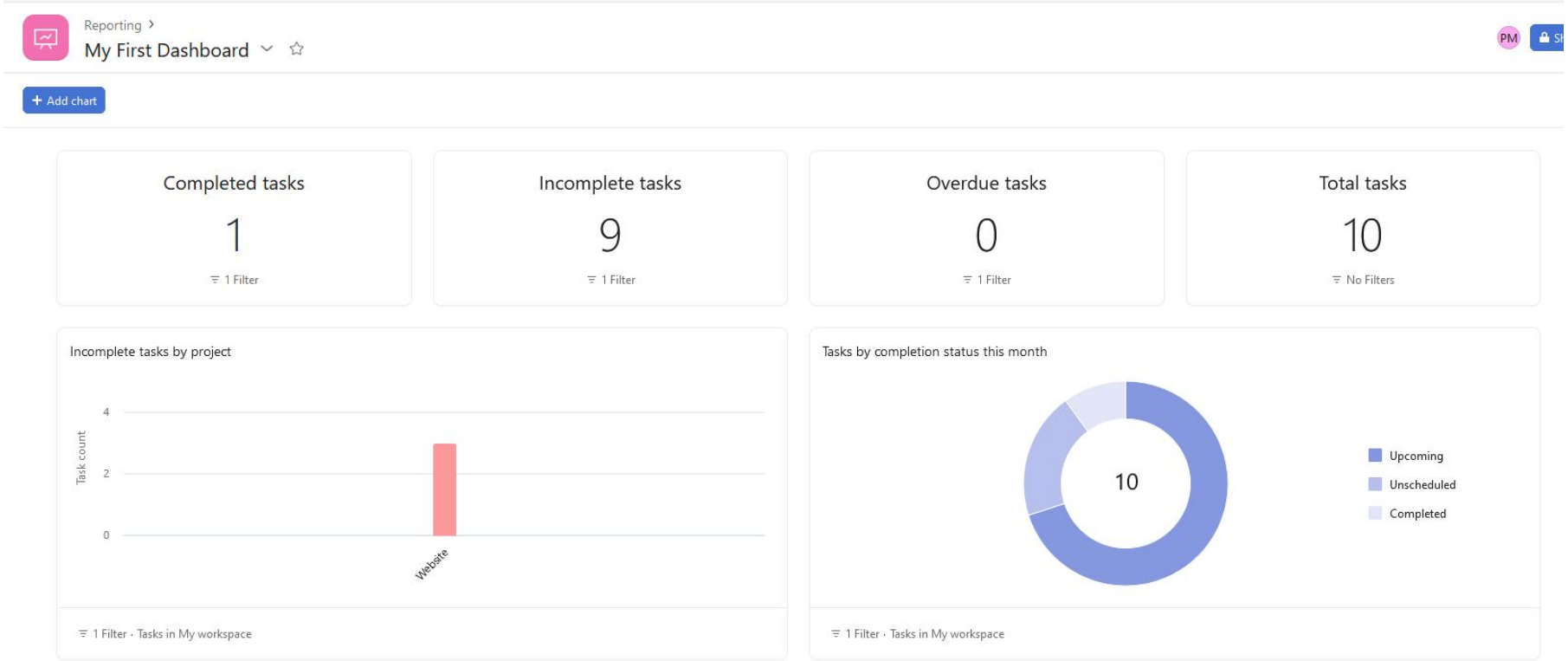
Add and Assign Tasks



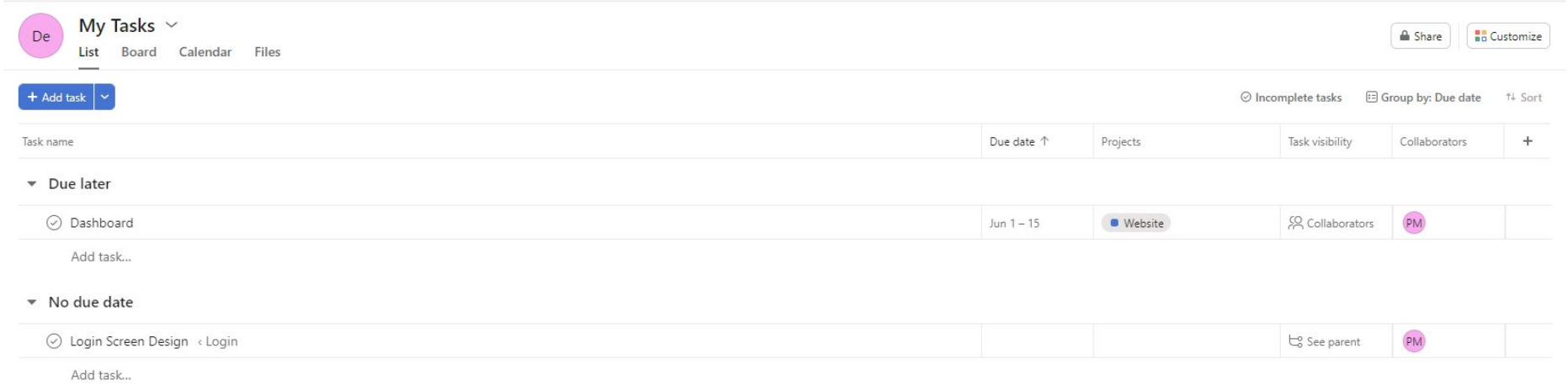
Set Timelines for a Task



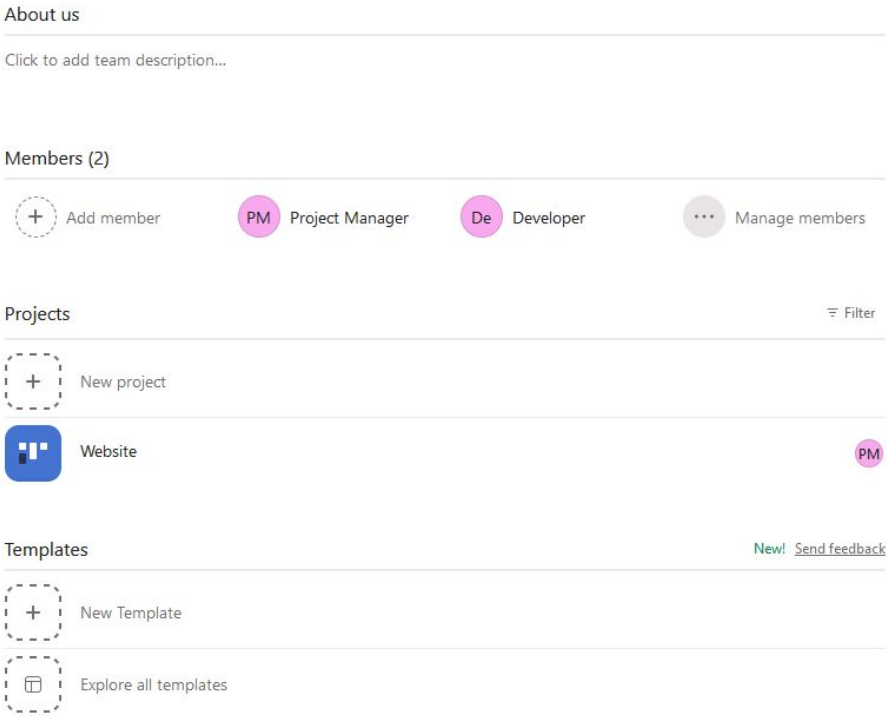
Project Manager Dashboard View



Developer View of Pending and Completed Tasks



Work Space and Team



Software construction and development Practical - I

Week 9

Good Class Design Guidelines

- **Encapsulation**: The class properly encapsulates its internal state by making member variables private and providing public getters and setters for controlled access and modification.
- **Information Hiding**: Internal implementation details are hidden from external code, reducing dependencies and improving code maintainability.
- **Single Responsibility Principle:** The class focuses on a specific purpose or functionality, ensuring that each class has a clear and well-defined responsibility.
- **Proper Constructors:** The class provides constructors to initialize its state appropriately, ensuring that objects are created in a valid and consistent state.
- **Error Handling:** The class handles potential errors gracefully, such as division by zero, by using exception handling and providing informative error messages.
- **Flexibility and Extensibility:** The class is designed to be easily extended or modified in the future without impacting existing code.
- **Meaningful toString:** The class overrides the `toString` method to provide a meaningful string representation of the object, facilitating debugging and logging.

Bad Class Design Guidelines

1. **Lack of Encapsulation:** The class exposes its internal state directly, making member variables public and allowing unrestricted access and modification from external code.
2. **Violation of Information Hiding:** Internal implementation details are exposed, leading to tight coupling and potential issues when modifying the class or its dependencies.
3. **Violation of Single Responsibility Principle:** The class performs multiple unrelated tasks, resulting in code duplication and decreased maintainability.
4. **Absence of Proper Constructors:** The class lacks constructors to initialize its state properly, potentially leading to objects with inconsistent or invalid states.
5. **Error Handling Neglect:** The class does not handle potential errors or exceptions, leading to unexpected behavior or runtime exceptions.
6. **Limited Flexibility and Extensibility:** The class design restricts the ability to extend or modify its functionality easily, leading to code duplication or the need for significant changes.
7. **Inadequate toString:** The class does not override the `toString` method, making it harder to obtain meaningful representations of objects for debugging or logging purposes.

Example Well designed class

```
public class Car {

    private String make;

    private String model;

    private int year;

    private double price;

    public Car(String make, String model, int year,
double price

        this.make = make;

        this.model = model;

        this.year = year;

        this.price = price;    }

 public String getMake() {return make;}

 public void setMake(String make) {
this.make = make; }

 public String getModel() { return model;}

 public void setModel(String model) {
this.model = model;  }

public String toString() { return "Car{" +

                "make='" + make + '\'' +

                ", model='" + model + '\'' +

                ", year=" + year +

                ", price=" + price +

                '}';  }
```

Example bad designed class

```
public class BadClass {

    public String name;

    public int age;

public void printName() {        System.out.println("Name: " + name);    }

public void printAge() {        System.out.println("Age: " + age);    }

  public void printNameAndAge

 System.out.println("Name: " + name + ", Age: " + age);

}    }
```

Example bad designed class

```
public class Calculator {
    public double result;
    public double add(double a, double b) {
        result = a + b;
        return result;
    }
    public double subtract(double a, double b)
{
        result = a - b;
        return result;
    }
    public double multiply(double a, double b)
{
        result = a * b;
        return result;
    }
    public double divide(double a, double b) {
        result = a / b;
        return result;
    }
}
```

Example bad designed class converted to well designed

```
public class Calculator {

    private double result;

  public double divide(double a, double b) throws
ArithmeticException{

if (b == 0) {

throw new ArithmeticException("Cannot divide by
zero.");}

 result = a / b;

        return result;

    }

   public double getResult() {

        return result;

    }

    public void setResult(double result) {

        this.result = result;

    }

}
```

Software construction and development Practical - I

Week 10

MISRA C++ coding standard

The MISRA C++ coding standard is a set of guidelines for writing safe, reliable, and maintainable C++ code. It was developed by the Motor Industry Software Reliability Association (MISRA) and provides a comprehensive set of rules and recommendations.

Rule 1: Maintain a consistent coding style

```
void calculateSum(int a, int b)
{
 int sum = a + b;
      cout << "Sum: " << sum << endl; }
```

Bad Practice

Good Practice

```
void calculateSum(int a, int b)
{
      int sum = a + b;
      cout<<"Sum: "<< sum <<endl;
}
```

Rule 2: Use the C++ Standard Library appropriately.

```
 include <vector>

 std::vector<int> numbers = {1, 2, 3, 4, 5};

for (const auto& num : numbers)

 {

std::cout << num << " ";

}
```

Rule 3: Avoid implementation-defined or undefined features.

Bad Practice

```
int a = -5;

unsigned int b = a;
```

Implementation-defined behavior

```
// Good: Using explicit type
conversion with defined behavior
```

Good Practice

```
int a = -5;

unsigned int b = static_cast<unsigned
int>(a);
```

Rule 4: Avoid dynamic memory allocation whenever possible.

```
// Bad: Manual dynamic memory allocation

int* array = new int[10];
```

Bad Practice

```
// Good: Using stack-based or static memory allocation

int array[10];
```

Good Practice

Rule 5: Minimize the use of explicit type conversions.

Bad Practice

```
float price = 19.99f;
 int quantity = 5;
 float total = static_cast<float>(quantity) * price;
```

Good Practice

```
float price = 19.99f;
int quantity = 5;
 float total = quantity * price;
```

Rule 6: Handle exceptions carefully.

Bad Practice

```
class Resource {

public: ~Resource()

 { if (!release()) { throw
std::runtime_error("Failed to
release resource.");

} } };
```

Good Practice

```
class Resource {

public: ~Resource() noexcept {

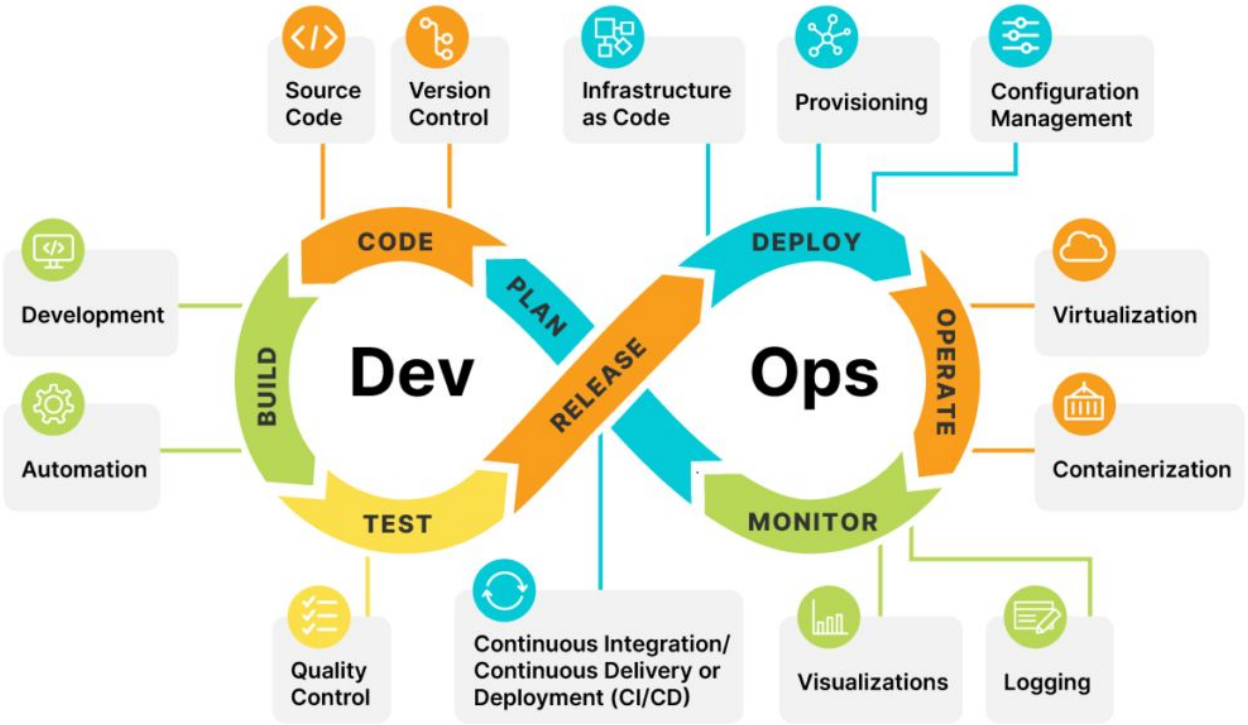
 release();

} };
```

Software construction and development Practical - I

Week 11

DEVOPS


Source Code
Version Control
Infrastructure as Code
Provisioning
Configuration Management
Development
Automation
CODE
PLAN
BUILD
Dev
RELEASE
DEPLOY
OPERATE
Ops
TEST
MONITOR
Virtualization
Containerization
Quality Control
Continuous Integration/ Continuous Delivery or Deployment (CI/CD)
Visualizations
Logging

Software Development Practices

- Programming Languages
- Version Control Systems
- Methodologies
- Software Development Life Cycle (SDLC)

Operating System and Networking

Learn system administration concepts such as operating systems, networking, server management, security, and virtualization technologies.

CI/CD

Explore and master key DevOps practices like continuous integration and continuous delivery (CI/CD), including tools like Jenkins, Travis CI, and GitLab CI/CD.

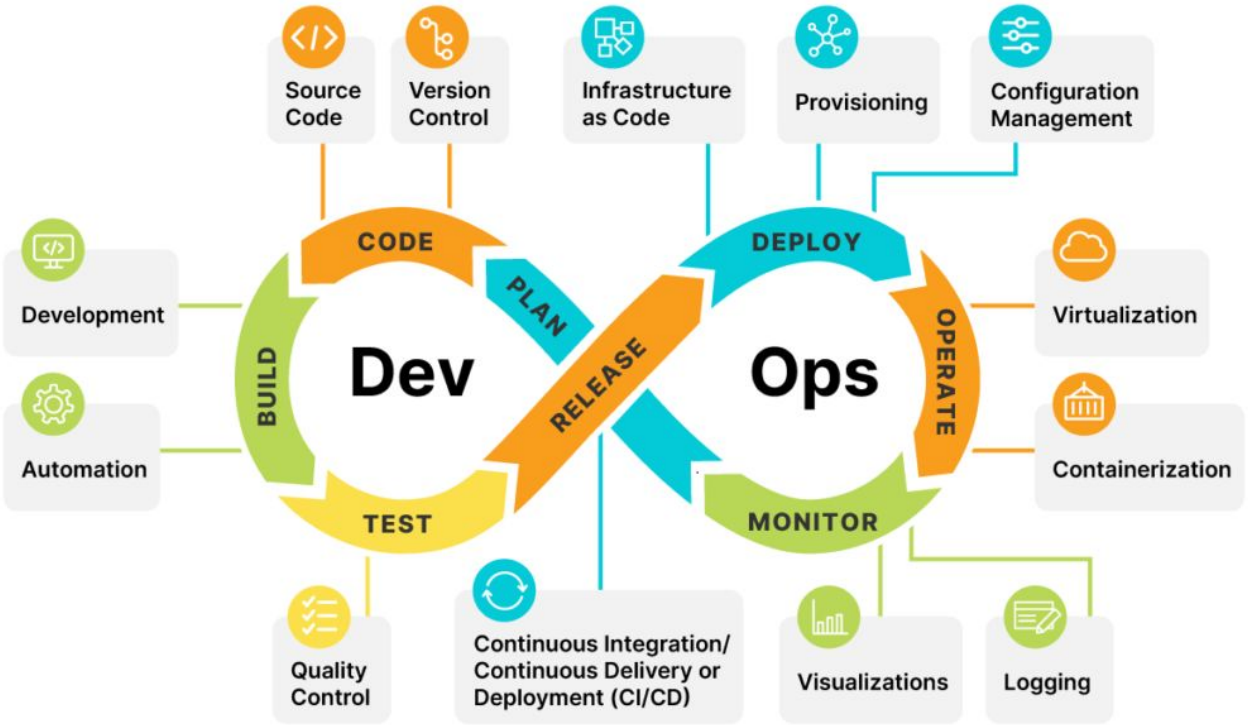
Essential DevOps tools

Acquire proficiency in essential DevOps tools and technologies, such as configuration management (Ansible, Chef, Puppet), containerization and orchestration (Docker, Kubernetes), cloud computing platforms (AWS, Azure, GCP), infrastructure automation (Terraform, CloudFormation), and monitoring/logging (Prometheus, Grafana, ELK stack).

Software construction and development Practical - I

Week 11

DEVOPS


Source Code
Version Control
Infrastructure as Code
Provisioning
Configuration Management
Development
Automation
CODE
PLAN
BUILD
Dev
RELEASE
DEPLOY
OPERATE
Ops
TEST
MONITOR
Virtualization
Containerization
Quality Control
Continuous Integration/ Continuous Delivery or Deployment (CI/CD)
Visualizations
Logging

Software Development Practices

- Programming Languages
- Version Control Systems
- Methodologies
- Software Development Life Cycle (SDLC)

Operating System and Networking

Learn system administration concepts such as operating systems, networking, server management, security, and virtualization technologies.

CI/CD

Explore and master key DevOps practices like continuous integration and continuous delivery (CI/CD), including tools like Jenkins, Travis CI, and GitLab CI/CD.

Essential DevOps tools

Acquire proficiency in essential DevOps tools and technologies, such as configuration management (Ansible, Chef, Puppet), containerization and orchestration (Docker, Kubernetes), cloud computing platforms (AWS, Azure, GCP), infrastructure automation (Terraform, CloudFormation), and monitoring/logging (Prometheus, Grafana, ELK stack).

Software construction and development Practical - I

Week 12

Heterogeneous Systems

Heterogeneous systems refer to computing systems that consist of different types of components or processors, often with varying architectures or capabilities. These systems combine different hardware or software elements to achieve specific goals or optimize performance in specific applications.

Diverse Components

Heterogeneous systems typically incorporate different types of components, such as CPUs (Central Processing Units), GPUs (Graphics Processing Units), DSPs (Digital Signal Processors), FPGAs (Field-Programmable Gate Arrays), or specialized accelerators. Each component is designed to excel at specific types of computations or tasks.

Performance Optimization

Heterogeneous systems leverage the unique capabilities of each component to optimize performance for specific workloads. For example, GPUs are highly efficient at parallel processing, making them suitable for tasks involving large amounts of data or complex computations.

Task Partitioning

In heterogeneous systems, tasks or workloads are divided among the different components based on their strengths and capabilities. This partitioning can be done at the hardware or software level, depending on the system architecture and programming models used.

Programming Challenges

Programming heterogeneous systems can be complex due to the diverse architectures and programming models associated with different components. Developers often need to use specialized APIs, libraries, or languages to effectively utilize the capabilities of each component and coordinate their interactions.

Application Areas

Heterogeneous systems find applications in various fields, including scientific computing, artificial intelligence, machine learning, image and video processing, data analytics, and high-performance computing. These systems enable faster and more efficient processing by leveraging the strengths of different components.

Energy Efficiency

Heterogeneous systems can also offer energy efficiency advantages by distributing tasks across components that are optimized for specific workloads. By utilizing components with different power profiles, these systems can achieve better performance-per-watt ratios compared to homogeneous systems.

System Integration

Integrating and managing heterogeneous systems require careful considerations for hardware compatibility, software interoperability, and system-level optimizations. This includes factors like data transfer between components, memory management, synchronization, and overall system performance tuning.

Software construction and development Practical - I

Week 13

GitLab

Your work / Projects / **New project**

Create new project



Create blank project

Create a blank project to store your files, plan your work, and collaborate on code, among other things.



Create from template

Create a project pre-populated with the necessary files to get you started quickly.



Import project

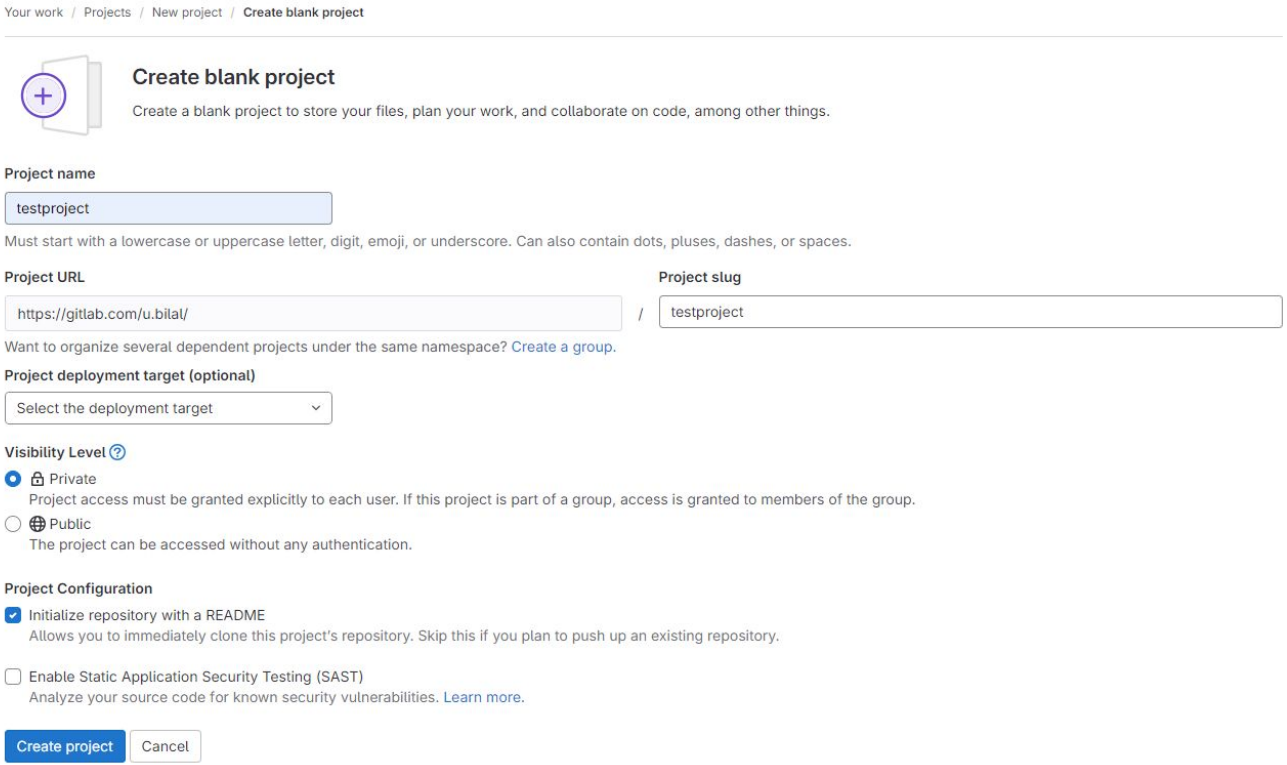
Migrate your data from an external source like GitHub, Bitbucket, or another instance of GitLab.



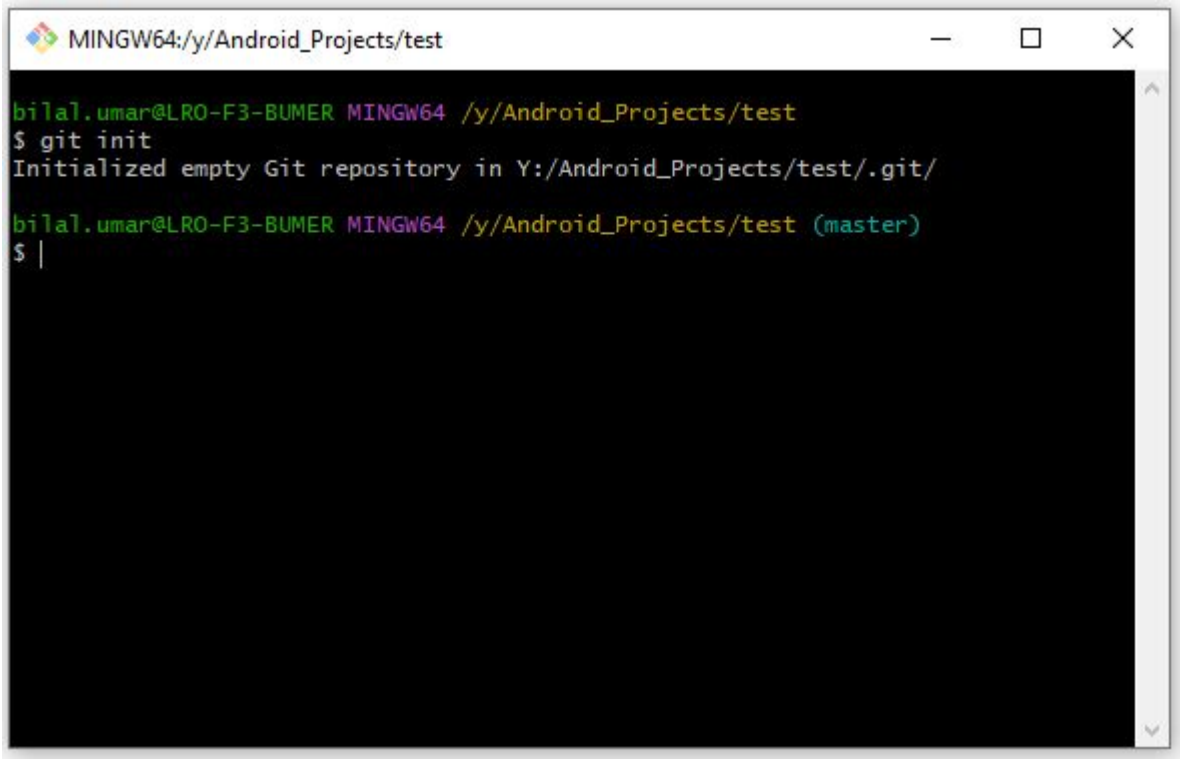
Run CI/CD for external repository

Connect your external repository to GitLab CI/CD.

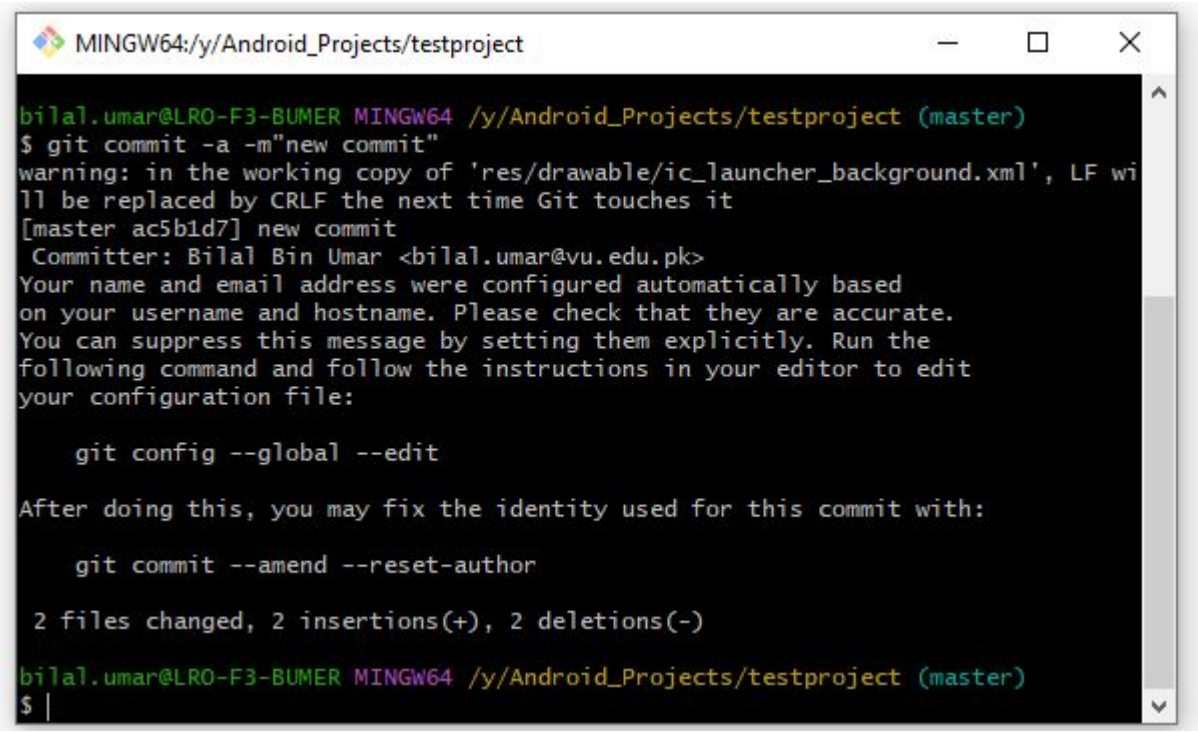
New Project


Your work / Projects / New project / Create blank project
Create blank project
Create a blank project to store your files, plan your work, and collaborate on code, among other things.
Project name
testproject
Must start with a lowercase or uppercase letter, digit, emoji, or underscore. Can also contain dots, pluses, dashes, or spaces.
Project URL
https://gitlab.com/u.bilal/
/
Project slug
testproject
Want to organize several dependent projects under the same namespace? Create a group.
Project deployment target (optional)
Select the deployment target
Visibility Level
Private
Project access must be granted explicitly to each user. If this project is part of a group, access is granted to members of the group.
Public
The project can be accessed without any authentication.
Project Configuration
Initialize repository with a README
Allows you to immediately clone this project's repository. Skip this if you plan to push up an existing repository.
Enable Static Application Security Testing (SAST)
Analyze your source code for known security vulnerabilities. Learn more.
Create project
Cancel

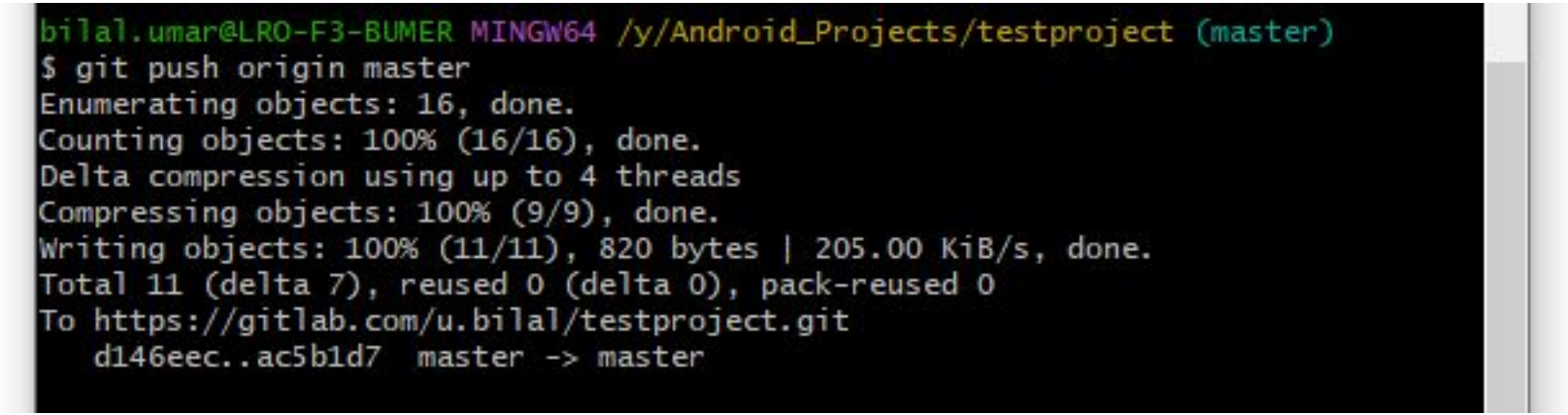
Git init


MINGW64:/y/Android_Projects/test
bilal.umar@LRO-F3-BUMER MINGW64 /y/Android_Projects/test
$ git init
Initialized empty Git repository in Y:/Android_Projects/test/.git/
bilal.umar@LRO-F3-BUMER MINGW64 /y/Android_Projects/test (master)
$

Git commit


MINGW64:/y/Android_Projects/testproject
bilal.umar@LRO-F3-BUMER MINGW64 /y/Android_Projects/testproject (master)
$ git commit -a -m"new commit"
warning: in the working copy of 'res/drawable/ic_launcher_background.xml', LF wi
ll be replaced by CRLF the next time Git touches it
[master ac5b1d7] new commit
Committer: Bilal Bin Umar <bilal.umar@vu.edu.pk>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:
git config --global --edit
After doing this, you may fix the identity used for this commit with:
git commit --amend --reset-author
2 files changed, 2 insertions(+), 2 deletions(-)
bilal.umar@LRO-F3-BUMER MINGW64 /y/Android_Projects/testproject (master)
$

Git Push


bilal.umar@LRO-F3-BUMER MINGW64 /y/Android_Projects/testproject (master)
$ git push origin master
Enumerating objects: 16, done.
Counting objects: 100% (16/16), done.
Delta compression using up to 4 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (11/11), 820 bytes | 205.00 KiB/s, done.
Total 11 (delta 7), reused 0 (delta 0), pack-reused 0
To https://gitlab.com/u.bilal/testproject.git
d146eec..ac5b1d7 master -> master

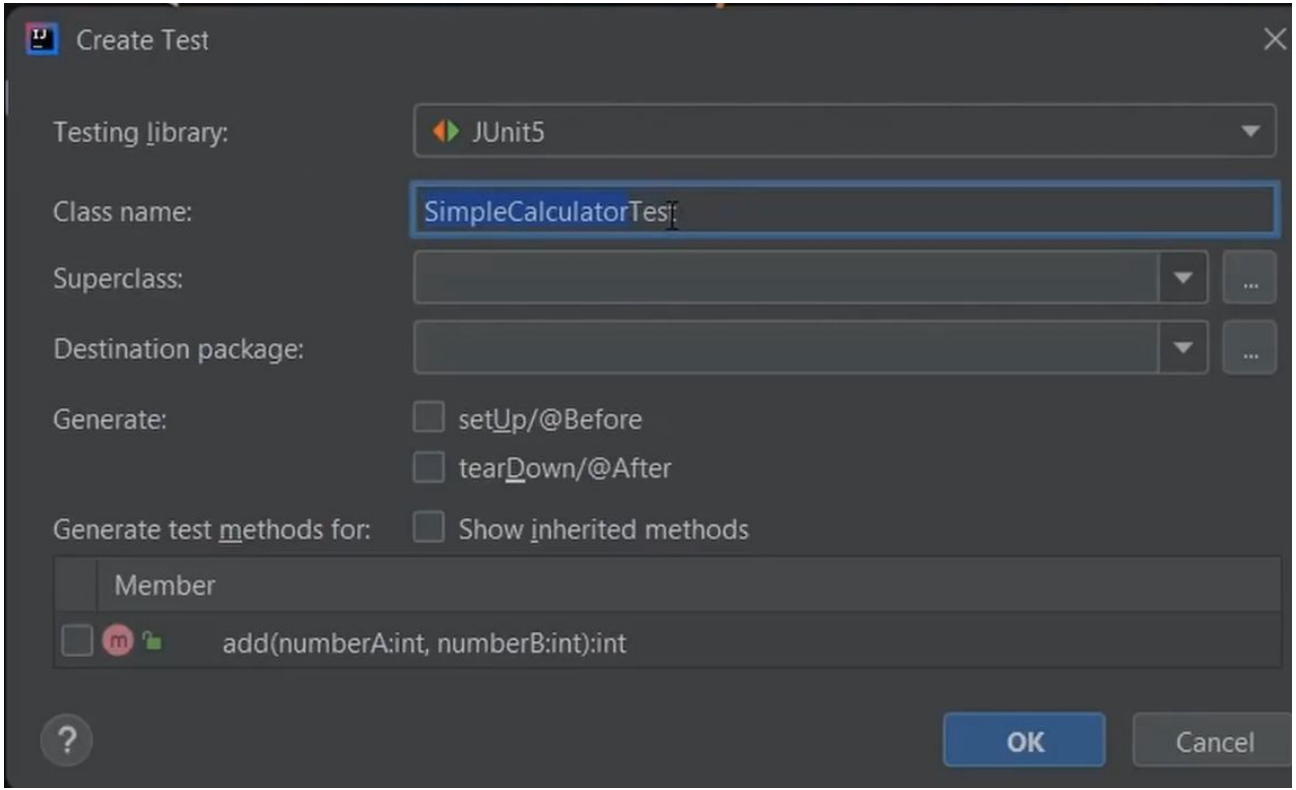
Software construction and development Practical - I

Week 14

Unit Testing

- Unit testing is a software testing methodology that focuses on testing individual units or components of a software application in isolation.
- Each unit, typically a function or method, is tested independently of the rest of the application.
- The purpose is to verify that each unit performs as expected and to identify and fix any defects early in the development process.
- Isolation is achieved using mock objects or stubs to simulate the behavior of dependencies.
- Unit tests are automated, allowing for easy and frequent execution through testing frameworks or tools.
- It helps ensure code correctness, improves software maintainability, and facilitates continuous integration practices.
- By catching issues early, it reduces overall development costs and enhances the reliability of the software system as a whole.

Unit Test in Junit5



Simple Unit Test

```
@Test
void twoPlusTwoShouldEqualFour(){
    var calulator = new SimpleCalculator();
    assertEquals(4,calulator.add(2,2));
}
```

Run unit test with coverage

– – –

"**Run with coverage**" in JUnit refers to executing unit tests while collecting code coverage data. JUnit, along with coverage tools like JaCoCo, tracks lines, branches, and statements covered during testing. A code coverage report shows the percentage of code exercised by tests, helping identify untested code areas. High coverage indicates thorough testing, but it doesn't guarantee defect-free code. Developers can improve test suites based on coverage insights, ensuring more robust and reliable applications.

Assert Methods in java

- `assertEquals(boolean condition)`: Asserts that both parameters given are equal.
- `assertNotEquals(boolean condition)`: Asserts that both parameters are not equal.
- `assertTrue(boolean condition)`: Asserts that the given condition is true.
- `assertFalse(boolean condition)`: Asserts that the given condition is false.
- `assertNull(Object object)`: Asserts that the given object reference is null.
- `assertNotNull(Object object)`: Asserts that the given object reference is not null.
- `assertThrows(expectedException, executable)`: Asserts that the specified executable (lambda or method reference) throws the expected exception.

Assert Throws Example

```
---
assertThrows(IllegalArgumentException.class
,
   () -> {
      grader.determineLetterGrade(-1);
         });
```

Software construction and development Practical - I

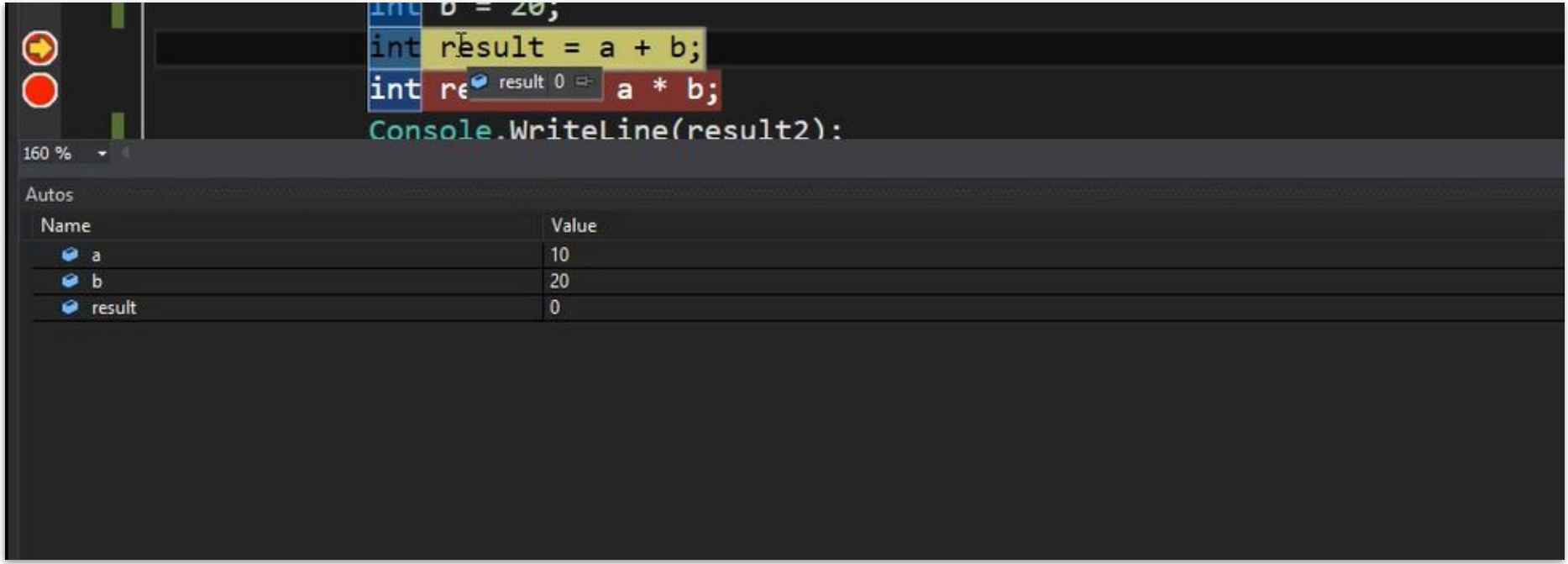
Week 15

Break points

– – –

In C#, breakpoints are used for debugging purposes, allowing you to **pause** the execution of your code at specific points to inspect the state of variables, check the flow of the program, and identify issues. To set a breakpoint in C#, you can use your IDE (Integrated Development Environment) like Visual Studio or Visual Studio Code.

Break Point Example



Break Point Types

- Standard Breakpoint
- Conditional Breakpoint
- Hit Count Breakpoint
- Tracepoint
- Function Breakpoint
- Data Breakpoint
- Exception Breakpoint

Debug Code

Console.WriteLine:

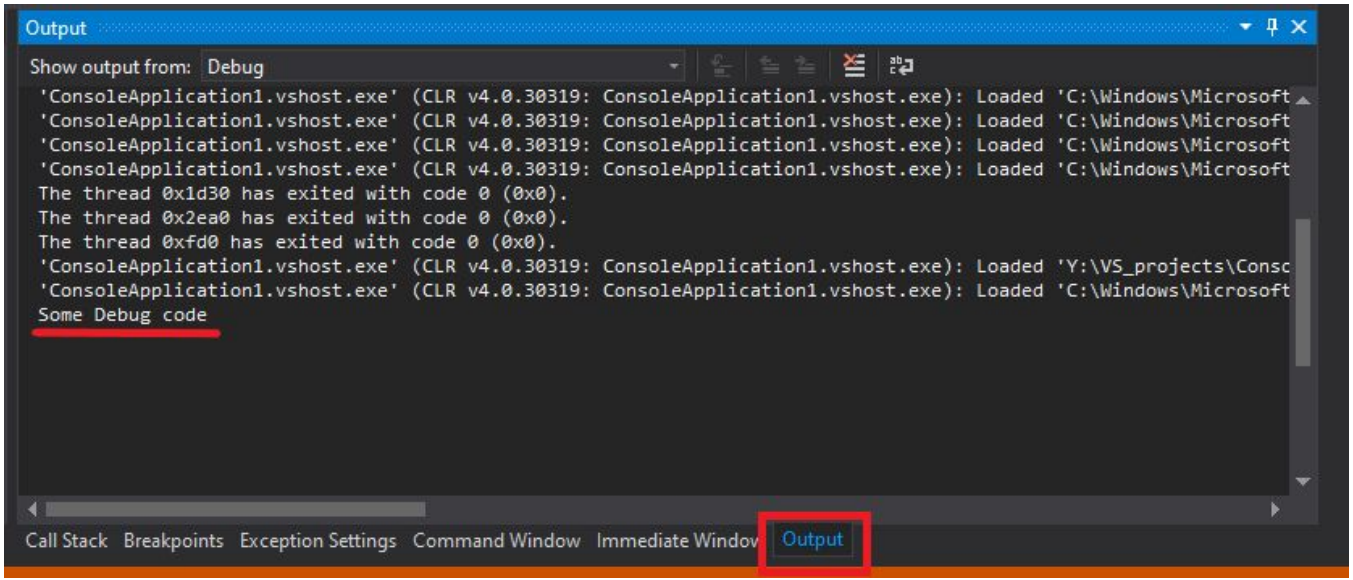
- **Console.WriteLine** is part of the **System** namespace and is used to output messages to the console, which is the standard output for console applications.
- Messages sent through **Console.WriteLine** will appear in the console window when running the application without debugging or when debugging with or without the debugger attached.
- It is typically used for regular application output, such as displaying information to the user.

Debug.WriteLine:

- **Debug.WriteLine** is part of the **System.Diagnostics** namespace and is used for outputting messages specifically during debugging.
- Messages sent through **Debug.WriteLine** will appear in the "Output" window of the IDE (e.g., Visual Studio) when running the application in debug mode.
- It is typically used for logging information, debugging insights, or tracking the program's flow and variable values while debugging.

Debug Code Example

```
Debug.WriteLine("Some Debug code");
```



Software construction and development Practical - I

Week 16

Automated Code Review

Automated code review, also known as static code analysis or automated code analysis, is a process that involves using software tools to automatically **inspect** and **analyze** source code to identify potential issues, vulnerabilities, and adherence to coding standards. This automated review helps ensure that the code meets specific guidelines, best practices, and security standards before it is merged into the main codebase or deployed.

Code Review Checks

- **Syntax and Style**: Checking for syntax errors and enforcing coding style guidelines.
- **Code Complexity**: Identifying overly complex or convoluted code that might be hard to maintain or debug.
- **Code Smells**: Detecting patterns that suggest potential design or implementation issues.
- **Security Vulnerabilities**: Identifying common security flaws like SQL injection, cross-site scripting, etc.
- **Code Duplication**: Spotting duplicated code, which can lead to maintainability issues.
- **Dependency Analysis**: Checking for dependencies and their versions to ensure security and compatibility.

Code Review Benefits

- **Consistency:** Automated code review ensures that all code is evaluated using the same set of rules and guidelines, leading to consistent code quality across the project.
- **Efficiency**: Manual code reviews can be time-consuming, especially for large codebases. Automated code review tools can quickly scan through the code, identifying potential issues much faster.
- **Early Detection of Issues**: Identifying issues early in the development process reduces the cost and effort required to fix them later.
- **Security**: Automated code review tools can help detect security vulnerabilities and prevent potential exploits, making the code more robust and less prone to attacks.
- **Enforcing Coding Standards**: Automated code review can ensure that developers follow coding standards and best practices, leading to more maintainable and readable code.
- **Integration with Development Workflow**: Many code review tools can be integrated directly into the development workflow, allowing developers to receive feedback and suggestions in real-time.
- **Scalability**: Automated code review scales well with large codebases and can handle an increasing number of code changes and submissions.

Code Review Tools Examples

- **ESLint**: A popular static code analysis tool for JavaScript that enforces coding standards and identifies common errors.
- **Pylint**: Widely used for Python projects, Pylint checks for programming errors, coding standards, and code quality analysis.
- **SonarQube**: A comprehensive platform supporting multiple programming languages for static code analysis, code coverage, and security vulnerability detection.
- **RuboCop**: A static code analyzer for Ruby that enforces the Ruby Style Guide and helps maintain consistent code formatting.
- **Checkstyle**: A tool for Java projects that enforces coding standards such as Sun Code Conventions, Google Java Style Guide, etc.

Code Review and Suggestions in Visual Studio

